ARCHITECTURE BREAKDOWN

Star Wars: Galactic Battlegrounds

Scenario Editor Architecture

An evidence-based reconstruction of the scenario and campaign authoring model behind Star Wars: Galactic Battlegrounds, derived from the SCX/CPX formats and independent open-source Genie tooling.

DOCUMENT	swgb.pdf
SCOPE	Editor architecture, data model, file formats, and engine lessons
EVIDENCE	Genie format documentation and five independent open-source readers
TARGET ENGINE	gdx-unity - Java 21 / LibGDX 1.14.2 / LWJGL3
DATE	27 October 2026

ANALYSIS LAYERS

Container and codec
SCX / CPX / DEFLATE / version profiles

Typed document
metadata / players / tiles / placements / triggers

Editor services
selection / commands / validation / identity

Editor UI
viewport / palettes / inspectors / dialogs

Runtime adapter
compile / spawn / trigger execution

Prepared for the gdx-unity documentation set. No retail Galactic Battlegrounds binary was analyzed.

CONTENTS

Report structure

01 1. Scope and Evidence 2

02 2. Executive Summary 3

03 3. Format Architecture 3

04 4. Document Model 6

05 5. Editor Workflow Reconstruction 9

06 6. Runtime and Editor Boundary 11

07 7. Architecture Patterns in the Tooling 12

08 8. Failure Modes and Invariants 14

09 9. Mapping to gdx-unity 15

10 10. Adoption Plan 19

11 11. Confidence and Open Questions 21

A Appendix A - Binary Quick Reference 22

B Appendix B - Evidence Index 23

C Appendix C - Glossary 24

1. Scope and Evidence

This report reconstructs the architecture of the Galactic Battlegrounds scenario editor from the files that editor produced and from independent open-source implementations that read those files. No retail Galactic Battlegrounds installation, executable, or scenario sample was present on the analysis machine. The retail editor UI was therefore not observed directly.

The analysis is still grounded in a strong artifact: the scenario file is the editor's persistence contract. Every panel, tool, and validation rule in the editor must ultimately produce a valid `.scx`, `.scn`, or `.cpx` file. Fields that exist only to support authoring - objectives, trigger display order, scenario hints, cinematic paths, player personality names, and a background bitmap - reveal the shape of the authoring model even when the original widgets are unavailable.

Confidence labels are used throughout:

- **Observed** - directly present in a format document or source implementation.
- **Inferred** - a reconstruction of editor behavior from persisted data or runtime behavior.
- **Unknown** - not established by the available evidence.

1.1 Evidence base

Source	Role in this analysis	Revision
`genieutils`	Authoritative C++ reader/writer for SCN, SCX, CPX; explicitly supports SWGB	`02baa4cc16e5e9e00425a0a1821f3b242cd9655d`
`genieutils/doc/scx_format.txt`	Community format specification for the shared AoE/SCX lineage	Same revision
`genie-scx`	Declarative JavaScript schema for SCX sections, with version branches	Repository snapshot
`AoE2ScenarioParser`	Mature typed document layer: sections, retrievers, managers, UUID store	Repository snapshot
`galactic-genie`	SWGB-specific clean-room CPX/SCX loader and runtime	Repository snapshot
`freeaoe`	Independent runtime consumer of the shared trigger model	Repository snapshot
`swgb-save-editor`	SWGB `.ga2` save editor; secondary evidence for tooling separation	Repository snapshot

The sources are not clones of the retail editor. They converge on the same binary layout from different directions: a generic C++ library, a schema-first JavaScript parser, a Python object model, a SWGB-specific C++ loader, and a runtime consumer. Agreement across those implementations is stronger evidence than any one of them alone.

1.2 What this report does not claim

- It does not claim the retail editor used the panel layout, command names, or undo model described here.
- It does not claim every SWGB release used the same version numbers or defaults.
- It does not claim the trigger editor was a free-form node graph. The file model is a two-level ordered list, not a general graph.
- It does not treat AoE2 behavior as automatically identical to SWGB behavior. Shared numeric trigger IDs can have different names and semantics.

2. Executive Summary

The Galactic Battlegrounds editor is best understood as a **section-oriented binary document editor with a game-data catalog attached**. It edits a sparse authored document, not a saved simulation. The document records initial placements, player configuration, terrain, and a trigger program; the game engine turns that into a running world at scenario start.

The most important architectural findings are:

1. **Two nested containers.** A `.cpx` campaign is a small archive whose entries are usually `.scx` scenarios. The campaign is a container document, not a special kind of scenario.
2. **One header, several incompatible dialects.** The scenario header `1.21` is shared by Age of Kings: The Conquerors and Galactic Battlegrounds. The internal layout differs, and `genieutils` needs an explicit `assumeSwgb` flag to choose the SWGB branch.
3. **The body is compressed as one stream.** The short header is uncompressed; the rest is raw DEFLATE with no stored uncompressed length. The editor must load and rewrite the whole body rather than patch arbitrary byte offsets.
4. **The document is positional, not self-describing.** Fields are gated by floating-point version numbers. Trigger effects and conditions carry a variable-length prefix of positional parameters. There are no field tags to recover meaning from.
5. **Most semantics live outside the file.** Unit constants, technology IDs, building footprints, terrain restrictions, garrison rules, and localized strings come from the game's DAT and string resources. The scenario stores IDs and placements, not definitions.
6. **Three reference graphs are unenforced.** Object spawn IDs, trigger indices, and string-table IDs connect different parts of the document. The binary format does not validate any of them. The editor must.
7. **Display order is not execution order.** Triggers, effects, and conditions each have a separate display-order array. Conflating the two silently changes what the author sees versus what the game runs.
8. **Terrain is a flat three-byte tile grid.** Terrain, elevation, and a third byte per tile, stored row-major. It is not a heightmap, mesh, or splat map.
9. **The authored/runtime boundary is real.** A scenario is a spawn recipe plus rules plus a trigger program. The runtime builds a world from it. The editor should not run the simulation in edit mode.
10. **The reusable architecture is layered.** Container codec, version profile, typed document, editor services, UI, and runtime adapter are separate concerns. The SWGB-specific parts belong in a game module, not in an engine core.

For `gdx-unity`, the strongest lesson is not "copy the SCX layout." It is "keep the authored document format-neutral and stable, isolate the binary adapter, and make identity, ordering, validation, and undo first-class editor services."

---

3. Format Architecture

3.1 Scenario container and compression

Observed. An SCN/SCX file begins with a four-byte ASCII version string and a 32-bit header length. The header that follows is uncompressed. Compression starts after the header and covers the rest of the file. The format document notes that no uncompressed length is stored, so decompression must be streamed in chunks. The compressed body ends with the script/AI section.

The header/body boundary is also the format's only natural transaction boundary. A writer that changes the header must recompute its length; a writer that changes the body must re-compress it. Keeping those operations separate makes both testable and prevents a header edit from silently invalidating the compressed stream.

Compression is also why the format has no random-access index. The section order is fixed, but section sizes are not stored, so a reader discovers boundaries by parsing forward. That is acceptable for a loader; it is hostile to a patcher.

The outer shape is:

```
+--------------------------------------+
| "1.21"                4 bytes        |
| header length         u32            |
| header                variable       |
+--------------------------------------+
| DEFLATE stream                       |
|   compressed header                  |
|   player data                        |
|   map                                |
|   unit blocks                        |
|   secondary player data              |
|   triggers                           |
|   included scripts / AI              |
+--------------------------------------+
```

The uncompressed header carries the save type, timestamp, scenario instructions, victory type, and enabled-player count. In `genieutils`, the computed header length on write is `21 + instructions.size()`, which is a useful reminder that this header is a fixed-size prefix plus one variable string.

Architectural consequence. A scenario editor cannot treat the file as a random-access database. It must decompress the body, build a document, edit the document, and serialize a new body. A tool that patches individual bytes can work for narrow save-editing tasks, but it is the wrong foundation for a scenario editor.

3.2 Version families and the SWGB ambiguity

Observed. The version table shared by the community format document is:

Header	Internal scenario version	Player-data version	Family
`1.10`	-	-	Age of Empires I
`1.18`	`1.13`	`1.18`	Age of Kings
`1.21`	`1.14`	`1.22`	The Conquerors
`1.21`	`1.15`	`1.30` (commonly)	Galactic Battlegrounds

The second and third columns are structural branch values used by readers, not alternate file-header strings. `genieutils` sets the internal scenario version to `1.15` only when `assumeSwgb` is true:

```
} else if (scn_ver == "1.21") {
    if (assumeSwgb) {
        scn_internal_ver = 1.15f;
    } else {
        scn_internal_ver = 1.14f;
    }
}
```

`genie-scx` likewise branches on player-data version `1.30` for 60 technology slots, 60 unit slots, and 60 building slots, while earlier versions use 30/30/20. `galactic-genie` treats the scenario as SWGB internally and reads a player-data version from the file, rejecting values outside a sane range.

The ambiguity is architectural, not cosmetic. The header alone cannot tell a reader whether `1.21` means The Conquerors or Galactic Battlegrounds. `genieutils` works around this with an out-of-band flag and a heuristic for campaigns: if an entry's filename begins with the campaign name, the entry is "probably SWGB." That heuristic is evidence that the format has no explicit game tag.

Recommendation. A production editor should not rely on a filename heuristic. It should:

1. carry an explicit format profile selected by the user or by a trusted project setting;

2. probe structure only as a fallback, with bounded reads and clear failure;
3. store the profile in the project metadata so reopening is deterministic;
4. keep the profile separate from the document data, because the same logical scenario may be exported to multiple dialects.

3.3 Campaign container

Observed. A CPX file is:

```
version        4 bytes, ASCII ("1.00" in the SWGB reader)
name           256 bytes, fixed
file count     u32
entries        count * 520 bytes
payloads       at absolute offsets
```

Each 520-byte entry is:

Field	Size	Meaning
`size`	4	Payload length
`offset`	4	Absolute offset from file start
`identifier`	255	ASCII identifier, often a scenario name
`filename`	257	ASCII filename, including extension

`galactic-genie` validates that the entry table fits inside the file, that each payload range starts after the table, that the range does not overflow, and that the version is `1.00`. `genieutils` performs the same read but does not enforce all of those bounds.

Observed consequence. CPX is uncompressed as a container; each embedded SCX carries its own compressed body. Saving a campaign requires rewriting the entry table and recomputing every payload offset. It is not enough to update a single entry in place when sizes change.

Inferred consequence. The campaign editor is a separate mode from the scenario editor. It needs entry add/replace/remove/reorder, filename uniqueness checks, and a launch action that opens an entry as a scenario. The binary table supports arbitrary embedded files, but the practical payload is a scenario.

3.4 Section pipeline

Observed. The decompressed body follows a stable order. The exact field set varies by version, but the sections do not:

Section	Contents	Primary editor concern
Compressed header	Next object ID, version, player names, player metadata, original filename	Identity allocation, player slots
Player data	Messages, cinematics, bitmap, resources, diplomacy, disables, AI names	Scenario settings, players
Map	Camera, map type, width/height, tile grid	Terrain editing
Units	Per-player object blocks, duplicate resources, population	Object placement
Secondary player data	Player names, cameras, diplomacy, colors, victory data	Player and victory settings
Triggers	Trigger system version, objectives state, triggers, display order	Trigger authoring
Scripts / AI	Included AI and personality files	Script attachment

The player data section is internally layered. The format document labels several blocks "Player Data 1" through "Player Data 4"; `genieutils` models them as `ScnPlayerData`, `ScnMorePlayerData`, and `ScnPlayerResources`. The units section contains a duplicate copy of player resources, which means a writer must keep two representations consistent. This is a classic denormalization: convenient for the original loader, dangerous for an

editor.

Observed. Section markers use the constant `0xFFFFFF9D` in several places. `galactic-genie` checks these separators explicitly before resources, victory conditions, allied victories, and camera data. A parser that skips them loses a strong structural check.

---

4. Document Model

4.1 Aggregate shape

A format-neutral model that preserves the observed data can be represented as:

```
ScenarioDocument
  Metadata
  PlayerSlot[16]
  TerrainGrid
  ObjectPlacement[]
  TriggerGraph
  EmbeddedScripts
  FormatProfile
```

This is deliberately not the binary layout. It is the smallest model that keeps authored meaning while allowing different codecs.

4.2 Metadata

Observed. The document persists:

- scenario instructions, hints, history, scouts text, and victory/loss text;
- original filename and save timestamp;
- pre-game, victory, and loss cinematic filenames;
- background bitmap (optional, with palette and pixel data);
- enabled player count and victory type;
- string-table IDs for instructions, hints, victory, loss, history, and scouts;
- localized player names and AI/personality names.

The presence of both inline strings and string-table IDs is important. A scenario can carry text directly, or refer to a localized string resource. The editor needs a text model that can represent both and can display a resolved value without destroying the source form.

Inferred UI. A "Scenario" or "Scenario Properties" panel with text areas for instructions/hints/history, file pickers for cinematics and bitmap, and read-only metadata. The large number of narrative fields suggests the original editor exposed them as tabbed or scrolling property pages rather than a single dialog.

4.3 Players

Observed. The scenario reserves 16 player slots; eight are playable in SWGB. Each slot carries:

- active/enabled and human/AI flags;
- civilization ID;
- initial resources (food, carbon, ore, nova, goods in SWGB terms; the binary slots are generic);
- population limit;
- starting age;
- AI type, AI filename, city filename, and personality filename;
- initial camera position;
- player color;

- diplomacy stances against all 16 slots;
- disabled technologies, units, and buildings;
- per-player victory conditions and alliance flags.

The diplomacy matrix is 16 x 16. The disable lists are per-player arrays sized by version; SWGB-era layouts use up to 60 technology slots and 60 unit slots, while older layouts use 30. This is a direct example of why version profiles must be data, not scattered `if` statements.

Inferred UI. A player table with rows for slots and columns for active, human, civilization, resources, and color; a separate diplomacy matrix; and a per-player tech/unit/building disable editor. The disable lists are long enough that they would need searchable checklists or tree views, not raw numeric entry.

4.4 Map and terrain

Observed. The map section stores:

- camera position;
- map type;
- width and height;
- a row-major grid of three-byte tiles: terrain ID, elevation, and a third byte.

`genie-scx` names the third byte `zone`; the older format document calls it unused. `galactic-genie` reads it and does not interpret it. The safest editor behavior is to preserve it and expose it only through a diagnostic or advanced field, never to zero it during a normal save.

The tile count is `width * height`. `genie-scx` models it as `height` arrays of `width` tiles, which makes the row-major layout explicit. There is no per-tile heightmap, normal, texture blend, or splat weight. Elevation is a discrete tile attribute, and rendering is a presentation concern.

Architectural consequence. Terrain editing is a grid operation, not a mesh-sculpting operation. The editor should store tile deltas, rebuild only affected render chunks, and derive passability from the game-data catalog rather than storing it in the tile.

4.5 Object placements

Observed. Each unit block begins with a count and then 29-byte records:

Field	Type	Meaning
`x`, `y`, `z`	f32	World position
`id`	u32	Spawn/reference ID used by triggers
`unitConst`	u16	Unit type constant
`state`	u8	Initial state byte; exact values not fully documented
`rotation`	f32	Facing angle in radians
`frame`	u16	Initial animation frame
`garrisonedIn`	s32	Container object ID, or -1

The scenario header carries `nextUnitID`, so new placements can be assigned monotonically increasing IDs. The units section normally contains nine blocks: Gaia first, then eight playable players. `galactic-genie` accepts up to 16 blocks for robustness.

The record is deliberately minimal. It does not store hit points, armor, attack, population cost, footprint, terrain restriction, or production prerequisites. Those come from the DAT. It also does not store a stable UUID beyond the 32-bit spawn ID. The ID is the only durable link between a placement and a trigger.

Inferred UI. Object placement uses a catalog browser or palette grouped by player and unit class, a viewport ghost, rotation controls, and an owner selector. Because the record has no footprint, the editor must query the

catalog at placement time for the preview and for validation.

4.6 Triggers

Observed. The trigger section begins with a trigger-system version (a double) and an objectives-state byte, followed by a count, the triggers, and a display-order array.

A trigger record contains:

Field	Meaning
`startingState`	Enabled at scenario start
`looping`	Re-arm after firing
`stringTableID`	Localized name/description reference
`isObjective`	Show in the objectives panel
`descriptionOrder`	Ordering within objectives
`startingTime`	Present in later trigger versions
`description`, `name`	Inline strings
`effects[]`	Ordered effect records
`effectDisplayOrder[]`	Editor display order
`conditions[]`	Ordered condition records
`conditionDisplayOrder[]`	Editor display order

An effect or condition is a type tag followed by a variable number of positional 32-bit fields, two length-prefixed strings (effects only), and an optional list of selected unit IDs. The count is a prefix-compression device: `genieutils` writes only as many trailing fields as are non-default, and reads the count from the file. Conditions use up to 16 fields; effects use up to 23. Older trigger versions use fixed counts of 13 and 16.

The fields cover the union of all trigger types: AI goal, amount, resource, diplomacy, selected-object count, object/location IDs, unit type, source and target player, technology, string ID, sound resource, timer, target trigger, point, area corners, object group, object type, bounded value, message, sound file, and selected unit IDs.

Observed consequence. The trigger model is a **typed record list**, not a generic property bag. The type tag determines which fields are meaningful and how they should be edited. A text field and a unit-type field can occupy the same numeric slot in different effect types. An editor that exposes raw field indices will produce nonsensical UI and easy corruption.

Observed. Trigger names are noted as limited to 44 characters in the UI by the community format document. That is a UI constraint leaking into the format documentation, and a reminder that the editor had a dedicated trigger-name field separate from the description.

4.7 Reference graph

The document contains several independent reference systems:

Reference	Source	Target	Enforced by format?
Object ID	Trigger condition/effect, garrison link	Unit placement	No
Trigger index	Activate/deactivate effects	Trigger list position	No
String ID	Trigger name/description, scenario text	String table	No
Unit/tech/building ID	Placement, disables, effects	DAT	No
Player ID	Most records	Player slot	Partially, by array bounds
Area/point	Conditions/effects	Map coordinates	No

This is the central editor-integrity problem. The binary format is a serialization of arrays, and arrays do not

enforce cross-references. Deleting a unit can leave a trigger pointing at a dead ID. Reordering triggers can make an "activate trigger" effect point at the wrong trigger. Renaming a string table entry can detach a description.

A robust editor needs a reference layer with stable internal identities, plus a validation pass that reports dangling references before save. It also needs deterministic remapping when the user reorders or deletes items.

---

5. Editor Workflow Reconstruction

This section is **inferred** from the persisted fields and from the runtime behavior those fields drive. The retail UI was not observed. The purpose is to identify the editor services the data model requires, not to claim a specific widget layout.

5.1 Shell and modes

A plausible reconstruction is a main editor shell with:

- a menu bar for file, edit, view, scenario, and help;
- a toolbar for save, test, undo/redo, and tool modes;
- a viewport with camera pan/zoom and a minimap;
- a palette or browser for terrain and objects;
- a properties panel for the current selection;
- a status bar for coordinates, player, and validation messages;
- modal or tabbed editors for players, triggers, and campaign entries.

The format implies at least four modes:

1. **Scenario editing** - terrain, placements, players, triggers.
2. **Campaign editing** - entries in a CPX archive.
3. **Test play** - launch a runtime copy of the scenario.
4. **Save/export** - validate, serialize, and write atomically.

The binary format has no undo journal, no dirty flags, and no selection state. Those are editor concerns. The retail editor may have had them; the file cannot prove it.

5.2 Map editing

Required operations:

- paint terrain, fill regions, and replace terrain in a selection;
- raise/lower or set elevation per tile;
- resize the map and define edge behavior;
- place player start objects and camera positions;
- preview passability and buildability from the catalog;
- render a minimap from terrain colors;
- maintain a camera bookmark for the scenario's initial view.

The editor should treat a brush stroke as one undoable transaction containing a list of tile deltas. Rebuilding the entire terrain mesh on every tile change is unnecessary; chunked rebuilds are sufficient. The tile grid is the source of truth; the mesh, minimap, and collision queries are derived.

5.3 Object placement

Required operations:

- choose a unit/building from a catalog filtered by player civilization and availability;

- preview footprint, clearance, terrain restriction, and elevation rule;
- place, move, rotate, retype, change owner, and delete;
- multi-select, box-select, copy/paste, and duplicate;
- edit initial state, animation frame, and garrison container;
- allocate new object IDs without reusing deleted ones;
- preserve IDs when moving or restyling an object, because triggers may reference them.

Placement validation should be shared with the runtime. `galactic-genie` documents the original building rules: clearance can differ from collision size; every footprint tile must satisfy terrain restrictions; side terrain can be required; elevation mode can require a level footprint or allow one step; map edges and obstructions invalidate placement. An editor that uses a simpler rule than the runtime will produce scenarios that fail on start.

Copy/paste needs reference remapping. If a copied group contains a garrison link or a trigger reference to one of its members, the pasted group should receive new IDs and rewrite internal references, while external references remain pointed at the originals unless the user explicitly retargets them.

5.4 Trigger authoring

The natural UI is a two-pane editor:

- left: trigger list, with enabled state, name, objective flag, and display order;
- right: selected trigger's conditions and effects, each with a type selector and a parameter form;
- bottom or side: validation and search.

Required operations:

- add/remove/reorder triggers, effects, and conditions;
- toggle enabled and looping;
- set objective display and order;
- choose condition/effect types from a filtered palette;
- edit typed parameters with pickers for players, units, technologies, areas, objects, and triggers;
- attach a sound and instruction text;
- copy a trigger for multiple players with controlled player-field rewriting;
- search by name, text, object ID, or type;
- validate before save.

The display-order arrays must be edited as presentation metadata, not as execution order. `AoE2ScenarioParser` makes this explicit in code: `trigger_display_order` is documented as "The display order. This is a list of trigger IDs in the display order. NOT execution order!" That comment is a warning earned by experience.

A trigger editor should also provide a dry-run or test harness. Trigger conditions depend on runtime state (objects in area, timer, technology researched, units garrisoned, power, population), so static validation cannot prove correctness. The practical workflow is: validate references, then launch a test scenario with debug overlays for trigger state.

5.5 Player and scenario settings

Required forms:

- scenario instructions, hints, history, scouts, victory, and loss text;
- cinematic and background file references;
- player slot table with active/human/civilization/resources/population/age/color;
- AI type, AI file, city file, and personality file per player;
- diplomacy matrix;

- technology, unit, and building disable lists;
- victory conditions and alliance flags;
- initial camera per player.

The diplomacy matrix and disable lists are the expensive parts. A 16 x 16 matrix is small, but a per-player list of 60 technologies, 60 units, and 60 buildings is a large structured selection problem. The UI needs search, select-all/none, civilization defaults, and a clear distinction between "disabled by scenario" and "unavailable to this civilization."

5.6 Campaign assembly

A campaign editor needs:

- an ordered list of entries with identifier and filename;
- add, replace, extract, remove, and reorder;
- duplicate filename detection;
- payload size and offset display for diagnostics;
- a launch action that opens an entry in the scenario editor;
- a save path that rewrites the table and all offsets.

Because the CPX table stores absolute offsets and fixed-size strings, campaign editing is more like writing a small archive than editing a document. The editor should build the entire archive in memory or a temporary file, then replace the original atomically.

5.7 Save and playtest

A safe save pipeline is:

1. validate the document;
2. resolve the format profile;
3. serialize to a temporary file;
4. re-read the temporary file and validate structure;
5. atomically replace the target;
6. keep a backup or previous version.

A safe playtest pipeline is:

1. validate;
2. serialize a temporary scenario or campaign;
3. launch the runtime with that file;
4. keep the authored document untouched;
5. on return, discard runtime state and restore selection.

The editor should never "test" by mutating the authored document into runtime state. That makes it impossible to distinguish authoring bugs from simulation side effects.

---

6. Runtime and Editor Boundary

A scenario is not a saved game. It is a set of initial conditions plus rules:

- terrain and elevation;
- player slots, resources, civilizations, diplomacy, and disables;
- object placements;

- trigger program;
- optional AI/personality scripts;
- presentation metadata (instructions, cinematics, bitmap).

The runtime consumes that document and builds a world. `freeaoe` shows the pattern clearly: it wraps each `genie::Trigger` in a runtime `Trigger` that tracks `enabled`, copies the effect list, and wraps each condition in a `Condition` with a mutable `amountRequired`. Timer conditions are converted from scenario seconds to runtime milliseconds. The runtime then decrements requirements and fires effects. The authored trigger is immutable input; the runtime trigger is mutable state.

`galactic-genie` shows the other half: a loader that validates counts, separators, finite floats, player ranges, tile dimensions, and trigger field counts before constructing runtime objects. Those checks belong in the editor as well, but they are not sufficient. The editor also needs semantic checks that the runtime loader cannot know: dangling object references, empty triggers, duplicate names, unreachable objectives, and invalid technology/unit combinations for the selected civilization.

The correct boundary is a **compile step**:

```
Authored document
    -> validate
    -> resolve catalog IDs and string IDs
    -> allocate runtime IDs
    -> build spawn plan
    -> compile trigger program
    -> runtime world
```

This is analogous to `gdx-unity`'s existing Play Mode design: the authored scene is serialized and reconstructed as an independent runtime copy. The scenario compiler should extend that pattern, not bypass it.

---

7. Architecture Patterns in the Tooling

7.1 `genieutils`: typed serialization with version switches

`genieutils` models the file as C++ classes with `serializeObject()` methods. It has a strong typed surface (`ScnFile`, `ScnPlayerData`, `Trigger`, `TriggerCondition`, `TriggerEffect`, `CpxFile`) and a central codec. This is a good demonstration of keeping binary layout in one layer and exposing a usable object model above it.

Its weaknesses are equally instructive:

- Version state is global/static (`scn_trigger_ver`, `scn_plr_data_ver`), which makes concurrent parsing and nested formats fragile.
- SWGB detection is a flag plus a filename heuristic, not an explicit profile.
- The write path computes a variable prefix length by scanning backward from the last field until a non-default value. That is clever, but it couples the writer to the exact default sentinel (`-1`) and to field order.
- The model preserves many unknown fields, but not as opaque named blobs; they are typed fields with guesses.

Lesson: centralize the codec and keep the model typed, but make the version profile an explicit parameter, not ambient global state.

7.2 `genie-scx`: schema as data

`genie-scx` describes sections with composable structs:

```
const MapData = struct([
  t.skip(4),
  ['camera', struct([['x', t.int32], ['y', t.int32]])],
  ['mapType', t.int32],
```

```
  ['width', t.uint32],
  ['height', t.uint32],
  ['tiles', t.array('height', t.array('width', struct([
    ['terrain', t.uint8],
    ['elevation', t.uint8],
    ['zone', t.uint8]
  ]))))]
])
```

Conditional fields are expressed as predicates over the current version. String mapping handles fixed-width null padding. The result is compact, readable, and easy to compare against a format document.

Lesson: a declarative schema is excellent for the byte layer and for generating documentation. It is not a semantic model. The schema knows that three bytes exist; it does not know that the third byte must be preserved, that terrain IDs come from the DAT, or that a trigger's field 7 means different things for different effect types.

7.3 `AoE2ScenarioParser`: sections, retrievers, managers

`AoE2ScenarioParser` is the most complete reference for a typed editor document layer:

- `AoE2FileSection` groups retrievers and structs for one section.
- `Retriever` is the smallest unit of data, with a datatype, default, dirty flag, and dependencies.
- `AoE2StructModel` describes repeatable records.
- `AoE2ObjectManager` constructs managers for messages, players, map, units, triggers, XS, and options, then commits them back to sections.
- `UuidList` and a scenario store give objects stable identity independent of array position.
- `TriggerManager` owns triggers, variables, display order, and operations such as copy-per-player.

The manager layer is a projection: it presents a user-facing object model over the raw section/retriever tree, and `commit()` writes changes back. This is a clean separation, but it is also a lot of machinery. For a smaller editor, a single typed document plus domain services is enough; the section/retriever layer is only necessary if byte-level fidelity and unknown-field preservation are hard requirements.

Lesson: separate raw storage from domain objects, but do not build a generic retriever framework unless you actually need to preserve arbitrary unknown fields. The important parts are stable identity, explicit dirty tracking, and a deliberate commit boundary.

7.4 `galactic-genie`: defensive SWGB loading

`galactic-genie` is SWGB-specific and clean-room. Its loader is notable for explicit limits and checks:

- maximum trigger fields (64);
- maximum trigger records (100,000);
- maximum units per player (20,000);
- player block count between 1 and 16;
- finite checks on positions, rotations, resources, and camera;
- separator checks;
- player-data version range check;
- exact trailing-data check after triggers.

These are not editor features, but they are the same invariants an editor should enforce before export. A scenario that cannot be loaded safely is not a valid save.

Lesson: validation is part of the format contract. Bounds and finiteness checks belong in the shared codec, not only in the runtime.

7.5 `freeaoe`: runtime consumption

`freeaoe` demonstrates that the authored trigger model and the runtime trigger model are different objects. The authored conditions are copied into wrappers with mutable counters. The authored effects are retained as an ordered vector. Trigger enablement is runtime state derived from `startingState`.

Lesson: keep simulation state out of the document. A trigger's "enabled" flag in the file is an initial condition; the runtime's enabled flag is mutable state.

7.6 `swgb-save-editor`: separate the save format

The SWGB save editor targets `.ga2` save files, not scenarios. It is useful mainly as a reminder that saves and scenarios are different artifacts with different lifecycles. A save is a snapshot of a running world; a scenario is an authored recipe. Mixing them into one document model produces confusing ownership and unnecessary fields.

---

8. Failure Modes and Invariants

The following invariants are derived from observed format behavior and from the failure modes visible in the parsers.

#	Invariant	Why it matters	Guard
1	A format profile is explicit	`1.21` is ambiguous between two games	Store profile in project; probe only as fallback
2	Every count is bounded before allocation	Malformed files can request huge arrays	Check count against remaining bytes and hard limits
3	Every offset is inside the file and after the table	CPX offsets are absolute and unvalidated by the format	Recompute and validate on read/write
4	Display order is separate from execution order	Reordering the UI must not reorder execution	Two arrays, two operations, explicit tests
5	Object IDs are unique and stable	Triggers reference them by 32-bit ID	ID allocator never reuses; deletion runs reference check
6	Trigger indices are remapped on reorder/delete	Activate/deactivate effects store an index	Stable trigger UUIDs plus compile-time remap
7	Areas are normalized or orientation is preserved	Lower-left/upper-right assumptions affect containment	Normalize on edit, preserve raw on import if needed
8	Selected-unit list length matches the count field	The count is a positional field inside the effect	One serializer owns both
9	String lengths are byte lengths, not character counts	Fixed buffers and `str16`/`str32` are byte-oriented	Encode explicitly; validate buffer capacity
10	Unknown fields are preserved	Zeroing "unused" data can break the original game	Opaque field preservation or canonical rewrite with tests
11	Write order is deterministic	Byte-for-byte diffs and reproducible builds	Ordered collections, stable sort
12	No silent truncation	Trigger names and fixed strings have UI/format limits	Validate before write; report the limit
13	Floats are finite and coordinates are in range	NaN/Inf can crash or corrupt runtime	Validate on edit and on load
14	Saves are atomic	A failed write destroys the only copy	Temp file, fsync, replace, backup
15	Playtest uses a serialized copy	Runtime mutation hides authoring bugs	Compile to temp file; restore authored state
16	Validation runs before export	Binary formats fail late and opaquely	Blocking errors, warning list, review dialog

8.1 The denormalization trap

The units section duplicates player resources and population. The player data section also stores them. If the

editor updates one copy and not the other, the scenario may load with different values depending on which section the runtime reads first. A typed document should have one resource model and the codec should write both copies from it.

8.2 The "unused byte" trap

The third map tile byte is called unused by the older format document and `zone` by `genie-scx`. If it is truly unused by SWGB, zeroing it is harmless; if it is used by a later release or by a specific map type, zeroing it is data loss. The safe default is preservation.

8.3 The "version branch" trap

Version checks scattered across a codec become contradictory. `genieutils` branches objectives state at `> 1.4`, while `genie-scx` branches at `>= 1.5`. That difference may be harmless for the versions actually encountered, but it illustrates the risk. A profile object should own the predicates:

```
interface ScenarioProfile {
    boolean hasObjectivesState(double triggerVersion);
    boolean hasStartingTime(double triggerVersion);
    int technologySlots();
    int unitSlots();
    int buildingSlots();
    int effectFieldCount(int triggerVersion);
    int conditionFieldCount(int triggerVersion);
}
```

The codec asks the profile; the profile is tested once.

---

9. Mapping to `gdx-unity`

`gdx-unity` is a general engine with a Unity-inspired editor, a versioned JSON scene format, stable entity UUIDs, a command/history system, and an authored/runtime Play Mode split. Those are the right foundations. The SWGB analysis should change how the project models authoring data, not turn the engine into a Genie clone.

9.1 Module ownership

Concern	Owner	Rule
Entity/component model, stable IDs, JSON scene	`engine-scene`	No SWGB or Genie types
Terrain rendering, chunk meshes, camera	`engine-render`	Consumes a tile-grid asset; does not own authoring rules
Selection, commands, validation, document services	`editor-core`	Generic editor mechanics; game rules plug in
Panels, viewport, dialogs, shortcuts	`editor-desktop`	Presentation only; no format logic
SWGB profile, DAT catalog, trigger types, importer/exporter	Game/tool module	Optional; must not leak into engine core
Campaign archive	Tool module or asset pipeline	Container concern, not scene concern

The most important rule is the last two rows. SWGB unit constants, technology IDs, and trigger semantics are game content. They belong in a `games/swgb` or `tools/swgb` module that produces canonical scenes/assets. The engine should be able to run a different game without compiling SWGB knowledge.

9.2 Canonical authored shape

The existing scene is the right composition root. A scenario can be represented as a scene with specialized components and assets:

```
Scene
  ScenarioRoot
    ScenarioMetadataComponent
    PlayerTableComponent
    TriggerGraphComponent -> TriggerGraphAsset
  Terrain
    TerrainComponent -> TileGridAsset
  Placements
    TransformComponent
    SpriteRendererComponent
    ScenarioObjectComponent
  Runtime-only entities (created in Play Mode)
```

`ScenarioMetadataComponent`, `PlayerTableComponent`, and `ScenarioObjectComponent` are game-module components, not engine components. They carry only authored data. The catalog (unit stats, footprints, terrain rules, localized names) lives in assets keyed by stable IDs.

This is preferable to a parallel `ScenarioDocument` that duplicates the entity list. It reuses the existing scene serializer, UUIDs, selection, inspector, commands, and Play Mode isolation. The trade-off is that a scenario-specific importer must translate SCX records into entities and components; that translation is exactly where game-specific rules belong.

9.3 Terrain

Recommended shape:

- `TileGridAsset`: width, height, tile records (terrain ID, elevation, flags/reserved byte).
- `TerrainComponent`: reference to the asset, tile size, origin, chunk size.
- `TerrainRenderer`: builds one mesh per chunk from the grid; rebuilds only dirty chunks.
- `TerrainEditingService`: brush, fill, elevation, resize; produces tile deltas.
- `TerrainQueries`: passability, buildability, elevation at a point; backed by the catalog.

Do not create one entity per tile. Do not store collision geometry in the tile. Do not build a smooth heightmap if the target style is discrete tile elevation. Start with the simplest representation that matches the game: a grid, a chunked mesh, and catalog queries. Add texture blending or autotiling only when the visual acceptance requires it.

Undo should record tile deltas, not full grid snapshots. A brush stroke is a composite command containing the changed tiles. Resizing the map is a separate command with explicit old/new dimensions and a policy for cropped or padded content.

9.4 Object placement

Recommended shape:

- `ScenarioObjectComponent`: source object ID, owner player, unit type ID, initial state, rotation/frame if not already in Transform/Sprite, garrison reference.
- `PlacementCatalog`: resolves unit type ID to footprint, clearance, terrain restriction, elevation mode, garrison compatibility, civilization availability, and display name.
- `PlacementValidator`: shared by editor preview and runtime spawn; returns structured diagnostics.
- `ObjectIdAllocator`: monotonic 32-bit IDs for import/export; UUIDs remain the engine identity.

Placement flow:

```
select catalog entry
  -> create preview entity (not in document)
  -> validate footprint against terrain + occupancy
  -> snap/rotate
  -> commit PlaceObjectCommand
  -> assign UUID + source object ID
```

Move and rotate should preserve both UUID and source object ID. Retype may preserve the source ID but must revalidate footprint and garrison links. Delete must run a reference check and offer to clear or retarget trigger references.

9.5 Triggers

Model the trigger system as a **typed rule list asset**, not a free-form node graph:

```
TriggerGraphAsset
  TriggerDefinition[]
    uuid
    enabledAtStart
    looping
    name
    description
    objective (flag, order, string)
    ConditionDefinition[]
      type
      typed parameters
    EffectDefinition[]
      type
      typed parameters
    displayOrder[]
```

Execution order is the array order of `TriggerDefinition`, `ConditionDefinition`, and `EffectDefinition`. Display order is a separate list of UUIDs or stable indices. References between triggers use UUIDs, not array positions. Compilation to runtime closures resolves UUIDs to runtime handles and remaps activate/deactivate targets.

This preserves the actual SWGB model and avoids overgeneralizing into a visual scripting system. A node graph can be a later view over the same typed model, but it should not be the storage format.

The typed parameter schema is the core of the editor UI. Each effect/condition type declares:

- a stable type ID and display name;
- parameter fields with names, types, defaults, and visibility rules;
- which fields are player, object, area, technology, unit, string, sound, or trigger references;
- which fields are meaningful only for certain trigger versions;
- validation rules (non-negative counts, area ordering, required message for display effects).

The schema belongs in the game module. The editor renders forms from it and does not hardcode one widget per effect type.

9.6 Campaigns

Model a campaign as a manifest asset, not a scene:

```
CampaignManifest
  name
  entries[]
    uuid
    identifier
    filename
    asset reference
    order
```

The manifest stores no byte offsets. A CPX exporter computes the table, payload sizes, and offsets at export time. An importer reads the table, validates ranges, and creates stable entry UUIDs. Reordering entries is a manifest edit, not a binary rewrite.

9.7 Commands and undo

Reuse `CommandHistory` and the existing command pattern. Add commands where the document changes:

Command	Payload	Notes
`PaintTilesCommand`	tile delta list	Coalesce a brush stroke into one command
`ResizeMapCommand`	old/new dimensions, crop policy	Preserve or explicitly discard content
`PlaceObjectCommand`	entity snapshot, source ID	Selection follows the new entity
`DeleteObjectsCommand`	full entity snapshots	Reference check before commit
`MoveObjectsCommand`	before/after transforms	Multi-select aware
`SetObjectOwnerCommand`	before/after owner	Revalidate civilization/catalog
`EditTriggerCommand`	before/after trigger graph or granular delta	Prefer granular for inspector edits
`ReorderTriggersCommand`	old/new order	Remap trigger references
`ImportScenarioCommand`	full document transaction	Undo restores the prior scene

Command rules:

- Commands operate on the authored scene, never on runtime entities.
- Getters do not mutate; no lazy ID allocation inside a getter.
- Undo restores selection and IDs, not just visual state.
- Composite commands are atomic; a failed sub-command does not leave a partial edit.
- Brush strokes coalesce by tool + target, not by time alone.
- Reordering commands remap references in the same transaction.

The existing Play Mode rule - block editing during Play/Pause - should remain. Scenario trigger state is runtime state and must not be undoable as an authored edit.

9.8 Validation

Add a `ValidationService` in `editor-core` with game-module rule providers. Diagnostics should carry:

```
code, severity, message, targetKind, targetUuid, fieldPath, quickFix?
```

Suggested categories:

Category	Examples	Severity
Structural	Missing terrain asset, invalid map size, duplicate entry filename	Error
Referential	Trigger references missing object; garrison target missing; string ID unresolved	Error
Semantic	Empty trigger, no effects, objective order collision, disabled unit also placed	Warning/Error
Catalog	Unknown unit type, technology not available to civilization, footprint does not fit	Error
Platform	Name exceeds limit, too many objects, coordinate outside map	Error
Style	Duplicate trigger names, unused variables, unreachable objective	Warning

Validation should run on demand, on save, and incrementally after edits. Results must be deterministic and selectable: clicking a diagnostic selects the offending entity, trigger, or tile. Export is blocked on errors; warnings require acknowledgment.

9.9 Play mode

Extend the existing Play Mode controller with a scenario compile step:

1. validate the authored scene;
2. serialize a runtime copy (existing pattern);
3. compile the trigger graph and spawn plan;
4. instantiate terrain, players, and placements in deterministic order;

5. start the trigger system only after the world is initialized;
6. expose debug overlays for trigger state, object IDs, and player resources;
7. on stop, discard the runtime scene and restore the authored scene and selection.

The compiler should be a pure function from authored data to a runtime plan, so it can be tested without a window. Runtime-only components should be stripped or ignored by the editor inspector.

9.10 Serialization

Keep the existing versioned JSON scene format. Add:

- a schema version and migration path for game-module components;
- stable enum encoding (string IDs or stable numeric IDs, not Java ordinal);
- ordered lists for triggers, effects, conditions, campaign entries, and display order;
- asset references, not absolute paths;
- optional `extensions` maps for importer metadata that must survive a round trip;
- atomic write and backup;
- deterministic field order for reproducible diffs.

Do not serialize derived data: world matrices, resolved catalog stats, runtime trigger counters, chunk meshes, or binary offsets. Do not make the SCX byte layout the project format. The binary format is an export target and an import source.

9.11 Testing

The smallest sufficient proof set:

Test	What it proves
Golden import	A known SCX/CPX maps to the expected document
Unmodified round trip	Import then export preserves bytes or semantic equality
Edited round trip	A scripted edit survives export and re-import
Malformed input	Truncated counts, bad offsets, NaN, huge arrays fail cleanly
Reference integrity	Deleting an object/trigger reports or remaps references
Ordering	Display order changes do not change execution order
Undo/redo	Apply/undo restores deep equality, including IDs and selection
Play isolation	Runtime edits never touch the authored scene
Determinism	Two saves of the same document are identical
Performance	Large map paint and chunk rebuild stay interactive

If a full SWGB importer is out of scope, the same tests apply to the canonical JSON document and to the terrain/trigger services.

---

10. Adoption Plan

This is a sequence of narrow, independently useful increments. It avoids building a SWGB importer before the editor has a document worth importing into.

Phase A - Authoring foundations

- `TileGridAsset`, `TerrainComponent`, chunked `TerrainRenderer`.
- `PaintTilesCommand`, `ResizeMapCommand`, tile-delta undo.

- `ValidationService` with structural and platform rules.
- Scenario compile step in Play Mode.

Exit criteria: paint a map, undo/redo a stroke, save/reload, enter Play Mode, and return to an unchanged authored scene.

Phase B - Scenario metadata and players

- `ScenarioMetadataComponent`, `PlayerTableComponent`.
- Scenario text, cinematics, bitmap references.
- Player resources, civilization, diplomacy, disables, AI references.
- Inspector panels and validation for player data.

Exit criteria: configure eight players, save/reload, and see the same values in Play Mode.

Phase C - Object placement

- `ScenarioObjectComponent`, `PlacementCatalog`, `PlacementValidator`.
- Place/move/rotate/retype/delete, multi-select, copy/paste.
- Object ID allocation and reference checks.

Exit criteria: place a mixed force, validate footprints, delete a referenced object, and resolve the diagnostic.

Phase D - Trigger authoring

- `TriggerGraphAsset`, typed condition/effect schemas.
- List/detail editor, parameter forms, reference pickers.
- Display order, objective metadata, copy-per-player.
- Trigger validation and debug overlay.

Exit criteria: author a multi-stage objective sequence, run it in Play Mode, and confirm execution order is independent of display order.

Phase E - Campaign container

- `CampaignManifest`, entry list, add/replace/reorder.
- CPX import/export adapter with offset validation.

Exit criteria: build a two-scenario campaign, export, re-import, and launch both entries.

Phase F - Optional SWGB compatibility

- SCX/SCN codec, DAT catalog adapter, trigger type mapping.
- Import/export CLI or tool window.
- Round-trip and malformed-input tests.

Exit criteria: import a real SWGB scenario, edit terrain and a trigger, export, and load it in the original game.

Phases A through E are engine/editor work. Phase F is the only phase that needs SWGB-specific knowledge, and it can be deferred or shipped as a separate tool.

Sequencing matters as much as scope. Each phase should end at a capability that a designer can exercise without the next phase, so the editor remains usable while the format layer is still incomplete. Each phase should also add validation at the boundary it introduces, so later phases inherit checked data instead of retrofitting checks onto an opaque document. The main risk is premature SWGB fidelity: tuning the document model to one retail build before the engine's own authoring needs are settled would import format accidents into the core.

---

11. Confidence and Open Questions

11.1 Confidence summary

Claim	Confidence	Basis
SCX body is DEFLATE after an uncompressed header	High	Format document and multiple readers
`1.21` is ambiguous between AoE2:TC and SWGB	High	`genieutils` explicit flag and comment
SWGB uses internal scenario version `1.15`	High	`genieutils`, `galactic-genie` comment
Player-data version `1.30` is common for SWGB	Medium	Format table and `genie-scx` branches; not observed in a sample
Map is a row-major 3-byte tile grid	High	`genie-scx`, format document, loader
Object records are 29 bytes with the listed fields	High	Format document and multiple readers
Trigger fields are positional and prefix-counted	High	`genieutils` serializer, `galactic-genie` reader
Display order is separate from execution order	High	Three independent implementations
CPX entries are 520 bytes with absolute offsets	High	`genieutils`, `galactic-genie`
SWGB has SWGB-specific trigger conditions/effects	High	`genieutils` enum aliases, runtime names
Retail editor panel layout	Low	Inferred only
Retail editor undo/validation behavior	Unknown	No binary or UI evidence
Exact meaning of unit `state` byte	Low	Readers preserve but do not fully interpret
Exact meaning of tile third byte	Low	Called unused and `zone` by different sources

11.2 Open questions

1. **Exact trigger-system version in SWGB.** The trigger block has its own double version. The branch thresholds are known, but the value used by SWGB was not observed in a sample file.
2. **Player-data version variance.** `1.30` appears in the format table and schema branches, but different SWGB editions or campaign scenarios may use other values. A sample corpus is needed.
3. **Unit state semantics.** The byte is preserved but not documented. Values may encode stance, garrison state, or production state. The editor should expose only validated values.
4. **Tile third byte.** "Unused" versus "zone" is unresolved. Preservation is the safe policy.
5. **CPX alignment and padding.** The reader validates ranges but does not document alignment requirements. The writer should follow observed alignment if a retail sample is available.
6. **CPX identifier semantics.** The 255-byte identifier and 257-byte filename are distinct fields. Their exact roles in the original campaign UI are not established.
7. **Editor-side placement rules.** The runtime rules are documented in `galactic-genie`; the editor may have used a preview that was more permissive. The safe choice is to share runtime validation.
8. **String resolution.** Inline strings and string-table IDs coexist. The precedence and fallback behavior in the retail game are not fully established.
9. **Object groups and selected-unit lists.** The fields exist, but the editor UI and the exact runtime semantics of all group types are not fully reconstructed.
10. **AI/personality editing.** The scenario stores filenames and embedded script content. Whether the retail editor edited those scripts or only referenced them is unknown.
11. **Clone Campaigns differences.** The analysis treats SWGB and Clone Campaigns as one family. There may be version or content differences not visible in the generic readers.
12. **Retail undo model.** No evidence exists for the scope, granularity, or persistence of undo in the original editor.

11.3 How to resolve

- Obtain a retail SWGB/Clone Campaigns installation and sample `.scx`, `.cpx`, and `.ga2` files.
- Capture the editor UI and record panel/tool behavior.
- Build a differential corpus: import each sample with two independent parsers, compare field values and unknown bytes.
- Export edited samples and load them in the retail game; treat load success and visual equivalence as acceptance.
- Instrument runtime trigger execution to confirm ordering, timer units, and objective behavior.
- Compare SWGB and Clone Campaigns samples to isolate version differences.

Appendix A - Binary Quick Reference

A.1 SCX/SCN

```
version             c4        "1.21" for SWGB
headerLength        u32
saveType            s32
lastSaveTime        u32
instructions        str32
victoryType         s32
enabledPlayers      u32
--- DEFLATE begins ---
nextUnitID          u32
playerDataVersion   f32
playerNames         16 * 256 bytes
playerNameStringIds 16 * u32
playerMeta          16 * (enabled, human, civ, unknown)
...
map                 width, height, tiles[height][width]
units               per-player blocks of 29-byte records
secondaryPlayers    names, cameras, diplomacy, colors, victory
triggers            version f64, objectives u8, triggers, display order
scripts             included AI/personality files
```

A.2 Map tile

```
terrain     u8
elevation   u8
zone/unused u8
```

A.3 Object record

```
x, y, z      f32
id           u32
unitConst    u16
state        u8
rotation     f32
frame        u16
garrisonedIn s32
```

A.4 Trigger record

```
startingState       s32
looping             s8
stringTableID       s32
isObjective         s8
descriptionOrder    s32
startingTime        s32   (later versions)
description         str32
name                str32
```

```
effectCount         s32
effects             effectCount * variable
effectDisplayOrder  effectCount * s32
conditionCount      s32
conditions          conditionCount * variable
conditionDisplayOrder conditionCount * s32
```

A.5 Effect/condition

```
type                s32
fieldCount          s32
fields              fieldCount * s32
message             str32   (effects)
soundFile           str32   (effects)
selectedUnitIds     field[4] * u32 (effects, when present)
```

A.6 CPX

```
version     c4
name        256 bytes
fileCount   u32
entries     fileCount * 520 bytes
payloads    absolute offsets
```

Appendix B - Evidence Index

Path	Used for
`genieutils/doc/scx_format.txt`	Header, sections, versions, map, units, triggers, CPX
`genieutils/include/genie/script/ScnFile.h`	Scenario and campaign model
`genieutils/include/genie/script/scn/Trigger.h`	Trigger/effect/condition fields and SWGB aliases
`genieutils/include/genie/script/scn/ScnPlayerData.h`	Player, diplomacy, resources, victory model
`genieutils/src/script/ScnFile.cpp`	Version branching, section order, CPX, SWGB flag
`genieutils/src/script/scn/Trigger.cpp`	Prefix field counts, strings, selected units
`genie-scx/src/struct.js`	Declarative schema, row-major tiles, version gates
`AoE2ScenarioParser/.../aoe2_file_section.py`	Section/retriever/struct construction and commit
`AoE2ScenarioParser/.../aoe2_object_manager.py`	Manager setup/reconstruct
`AoE2ScenarioParser/.../trigger_manager.py`	Display order, trigger operations, UUID lists
`AoE2ScenarioParser/.../retriever.py`	Smallest data unit, dirty flag, byte conversion
`galactic-genie/src/core/scenario.cpp`	SWGB loader, bounds checks, SWGB-specific fields
`galactic-genie/src/core/cpx.cpp`	CPX bounds and entry validation
`galactic-genie/docs/ORIGINAL_SYSTEMS.md`	Runtime placement, garrison, power, instruction semantics
`freeaoe/src/mechanics/ScenarioController.h`	Runtime trigger wrapper and condition counters
`swgb-save-editor/README.md`	Save-format tooling separation

B.1 Reproducibility notes

- `genieutils` revision `02baa4cc16e5e9e00425a0a1821f3b242cd9655d` (2021-07-14).
- The format document is community-maintained and explicitly incomplete ("This format is missing many new finds").
- The SWGB-specific loader is clean-room and validates structure but does not claim to reproduce every retail

behavior.

- No retail binary, installer, or scenario file was analyzed on this machine. All UI statements are inferred.

Appendix C - Glossary

Term	Meaning
SCN/SCX	Scenario file. SCN is the older extension; SCX is the compressed scenario used by later AoE/SWGB.
CPX/CPN	Campaign archive. CPX is the later container; CPN is the older campaign format.
DAT	Genie game-data file containing unit, building, technology, terrain, and civilization definitions.
Scenario	Authored initial conditions, map, placements, players, and trigger program.
Save	Snapshot of a running game, distinct from a scenario.
Trigger	Named rule with conditions and effects, optionally looping and objective-visible.
Effect	Ordered action executed when a trigger fires.
Condition	Predicate that must be satisfied for a trigger to fire.
Display order	Editor presentation order for triggers/effects/conditions; not execution order.
Spawn ID	32-bit object identifier used by triggers and garrison links.
String table ID	Reference to a localized string resource.
Profile	Explicit format/game variant used to select version branches and catalog rules.
Chunk	Fixed-size region of the terrain grid rendered as one mesh.
Authored document	Editor-owned data, as opposed to runtime simulation state.

End of report.